

ATIVIDADE DE PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

RELATÓRIO DA AULA 7: EXPRESSÕES, ATRIBUIÇÃO E CONTROLE

ALUNO:

Danilo Antonio Alves Rosa, 24047644-2

1. Introdução

A Aula 07 da disciplina Paradigmas de Linguagens de Programação aborda três elementos fundamentais para compreender como os programas são executados: expressões, atribuição e estruturas de controle. O material apresenta esses temas a partir dos capítulos 7 e 8 de Conceitos de Linguagens de Programação, de Robert W. Sebesta, 11ª edição. A proposta central é compreender não apenas como escrever uma expressão ou uma estrutura de controle, mas também como diferentes linguagens de programação interpretam o mesmo código.

A aula começa fazendo uma ligação com o conteúdo anterior. Depois de estudar o que as variáveis armazenam por meio dos tipos, o foco passa para a forma como os valores são combinados, armazenados e utilizados para determinar a ordem de execução das sentenças. Essa mudança é importante porque muitos erros em programas não acontecem por falta de lógica do problema, mas por detalhes da própria linguagem, como precedência de operadores, conversões automáticas, efeitos colaterais, escopo e comportamento dos comandos de seleção e repetição.

Outro ponto importante é a comparação entre linguagens. Exemplos em Python, Java, JavaScript, C, Go e Rust mostram que uma mesma construção pode produzir resultados diferentes dependendo das regras adotadas pela linguagem. Dessa maneira, a aula também trabalha a ideia de que o programador precisa conhecer as regras da linguagem utilizada e não pode depender apenas da aparência do código.

O material adota uma metodologia exploratória: antes de executar os exemplos, os estudantes são convidados a prever a saída. Depois, os códigos são executados no OneCompiler e os resultados são relacionados às regras apresentadas no livro. Essa

dinâmica é especialmente útil para demonstrar situações que parecem simples, mas possuem comportamentos inesperados.

Expressões, Atribuição e Controle

Aula 07 · Paradigmas de Linguagens de Programação

Engenharia de Software

Referência: Sebesta, Conceitos de Linguagens de Programação, 11ª ed., Caps. 7 e 8 (p. 299-361)

2. Objetivos da aula

O objetivo principal é entender como as linguagens avaliam expressões e controlam o fluxo de execução. Para o desenvolvimento de software, isso significa conseguir ler código de outras pessoas sem ser enganado pela indentação ou pela precedência, reconhecer conversões implícitas que podem esconder erros e escolher estruturas de controle adequadas para cada situação.

A aula também busca mostrar que decisões de projeto das linguagens têm consequências práticas. Uma linguagem pode optar por permitir mais liberdade ao programador, enquanto outra pode exigir conversões explícitas ou estruturas mais rígidas. Essas escolhas influenciam legibilidade, confiabilidade, facilidade de escrita e possibilidade de ocorrência de erros.

Como exemplos de problemas reais, o material apresenta o caso do “goto fail”, relacionado a uma linha duplicada de desvio no código da Apple; o travamento dos tocadores Zune em 2008 devido a um laço sem progresso; e problemas de comparação no PHP relacionados à coerção de tipos. Esses casos ajudam a mostrar que detalhes aparentemente pequenos podem ter consequências importantes.

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela endentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · “goto fail” (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

3. Metodologia e organização

A aula foi planejada para aproximadamente 80 minutos e está dividida em etapas. Nos primeiros minutos ocorre um quiz de abertura com pequenos trechos de código. Em seguida são estudadas expressões, precedência, efeitos colaterais e sobrecarga. A terceira parte trata de conversões, expressões booleanas e avaliação em curto-circuito. Depois aparecem as sentenças de atribuição e, por fim, as estruturas de controle, seleção, iteração, goto e comandos protegidos.

A dinâmica proposta para quase todos os exemplos segue quatro passos: apresentar o código, solicitar que a turma preveja a saída, executar o programa e explicar a regra da

linguagem que produziu aquele resultado. Esse método evita que a aula fique somente na memorização de conceitos e incentiva a observação do comportamento real dos programas.

Os exemplos foram organizados em uma pasta de exercícios e testados em diferentes ambientes do OneCompiler. Ao final, existe uma atividade prática de identificação de bugs, na qual o estudante deve descobrir a saída, identificar a regra responsável pelo problema e propor uma correção.

Roteiro da aula (80 min)



Tipo de aula: exploratória. A turma **prevê a saída antes de executar**.

Exemplos: pasta exemplos/, na ordem dos slides. Todos rodam no navegador em **onecompiler.com**.

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula - Sebesta Cap. 7

4. Expressões e avaliação de expressões

Uma expressão é formada pela combinação de operandos, operadores, parênteses e, em determinados casos, chamadas de funções. Avaliar uma expressão envolve obter os valores dos operandos e executar as operações definidas pelos operadores. Apesar de parecer uma tarefa simples, existem várias questões de projeto que influenciam o resultado.

Entre essas questões estão as regras de precedência, as regras de associatividade, a ordem de avaliação dos operandos, a possibilidade de efeitos colaterais, a sobrecarga de operadores e as misturas de tipos permitidas. Essas características variam entre linguagens e precisam ser conhecidas pelo programador.

Um exemplo apresentado é a expressão $a + b * c$, considerando $a = 3$, $b = 4$ e $c = 5$. Se fosse avaliada simplesmente da esquerda para a direita, o resultado seria 35. Se fosse da direita para a esquerda, seria 23. Entretanto, linguagens como Java e Python utilizam regras de precedência que fazem a multiplicação ocorrer antes da soma, resultando em 23.

O material também apresenta linguagens que adotam outras decisões. Em APL, por exemplo, a avaliação é feita da direita para a esquerda e existe um único nível de precedência. Em Lisp, a notação prefixa e os parênteses deixam explícita a estrutura da expressão, dispensando a necessidade de regras tradicionais de precedência.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	–
<code>1 + 2 + "3"</code>	<code>TypeError</code>	"33"	"33"
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)

Esquerda: `*` / `%` + `-`
Direita: `**` (Python, JS, Ruby, Fortran),
`++`, `--`, menos unário
Visual Basic: `^` à esquerda!

Excel

`=-2^2` dá 4: lá o menos unário vence o `^`.

Na dúvida, use **parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. · §7.2.1.1-7.2.1.3 · p. 301-305 · exemplos/01_precedencia

5. Precedência e associatividade

Precedência define qual operador deve ser considerado primeiro quando uma expressão possui operadores diferentes. Associatividade, por sua vez, determina a direção de avaliação quando operadores adjacentes possuem a mesma precedência. Na maioria das expressões comuns, operadores como soma, subtração, multiplicação e divisão associam à esquerda.

A exponenciação apresenta um exemplo importante de diferença entre linguagens. Em Python, `2 ** 3 ** 2` é interpretado como `2 ** (3 ** 2)`, produzindo 512. O menos unário também demonstra diferenças entre linguagens: em Python, `-2 ** 2` resulta em -4, pois a exponenciação possui precedência maior que o menos unário. No JavaScript, a forma `-2 ** 2` é considerada erro de sintaxe, exigindo o uso de parênteses.

O material destaca que os parênteses são uma maneira simples de deixar a intenção explícita. Mesmo quando o programador conhece as regras de precedência, o uso de parênteses pode facilitar a leitura por outras pessoas e reduzir ambiguidades.

Outro exemplo envolve a soma com texto. Em Java, a expressão `1 + 2 + "3" + 4 + 5` é avaliada da esquerda para a direita. Primeiro `1 + 2` resulta em 3; depois a presença da string faz o operador `+` atuar como concatenação, levando ao resultado textual 3345.

Em Python, uma operação equivalente entre inteiro e string gera erro de tipo.

A divisão e o resto também apresentam diferenças. Java e C realizam divisão inteira com truncamento em direção a zero, enquanto Python, utilizando `//`, arredonda para baixo. Assim, `-7 / 2` em Java produz -3 quando tratado como divisão inteira, enquanto `-7 // 2` em Python produz -4.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de `a` em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. - §7.2.2 - p. 307-309 - exemplos/02_efeitos_colaterais

6. Ordem de avaliação e efeitos colaterais

A ordem de avaliação torna-se especialmente importante quando uma expressão contém funções que alteram o estado do programa. O material apresenta uma função em C que modifica uma variável global e depois retorna um valor. Na expressão `a = a +`

fun1(), o resultado depende de quando o valor de a é lido em relação à execução da função.

Em Java, a linguagem garante avaliação da esquerda para a direita. No exemplo estudado, o resultado é 8. Já em C, o padrão não define a ordem de avaliação nesse caso, permitindo que o resultado observado dependa das decisões do compilador. O exemplo testado apresentou 20. Em Go, também existem regras específicas sobre a ordem, e o material chama atenção para a diferença entre a ordem das chamadas de funções e a leitura das variáveis.

Esse tema está relacionado à transparência referencial. Quando uma função não possui efeitos colaterais, a ordem de avaliação tende a deixar de ser relevante, pois substituir uma expressão pelo mesmo valor não altera o comportamento do programa. Funções puras, comuns em linguagens funcionais, favorecem esse tipo de previsibilidade.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("" + 'a' + 1);     // a1
x.add(y).multiply(tres);               // BigDecimal: sem +
```

Python - sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):           # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):               # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)         # Vetor(4, 6)
print(v * 3)                          # Vetor(12, 18)
```

7. Sobrecarga e conversões de tipos

Sobrecarga ocorre quando um mesmo operador pode possuir diferentes significados dependendo dos tipos envolvidos. O operador `+` em Java pode somar números e também concatenar strings. O exemplo `'a' + 1` produz 98, porque o caractere é convertido para seu valor numérico. Já `"" + 'a' + 1` produz `a1` porque a presença inicial de uma string direciona a operação para concatenação.

Algumas linguagens permitem que o próprio programador defina o comportamento de operadores. Python, por exemplo, utiliza métodos especiais como `__add__` e `__mul__`, permitindo que objetos como vetores sejam somados e multiplicados de forma semelhante aos números. Java não permite sobrecarga de operadores definida pelo usuário; por isso, classes como `BigDecimal` utilizam métodos como `add` e `multiply`.

As conversões podem ser de alargamento ou estreitamento. O alargamento leva um valor para um tipo com maior capacidade, enquanto o estreitamento pode provocar perda de informação. O material mostra que até mesmo uma conversão considerada de alargamento pode perder precisão, como no caso de um inteiro grande convertido para `float`.

Também é apresentada a diferença entre coerção e `cast`. Coerção é uma conversão implícita feita pela linguagem, enquanto `cast` é uma conversão explícita solicitada pelo programador. Java, C, Python, Go e Rust adotam estratégias diferentes. Go e Rust, por exemplo, exigem conversões explícitas em situações nas quais Java pode realizar uma conversão automaticamente.

JavaScript e PHP apresentam exemplos mais fortes de coerção implícita. Em JavaScript, `"5" - 2` resulta em 3, enquanto `"5" + 2` resulta em `"52"`. O operador `+` pode concatenar quando há texto, enquanto outros operadores forçam conversões numéricas. O material recomenda o uso de `===` e `!==` em JavaScript e PHP para evitar determinados problemas de coerção.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. - §7.4 - p. 310-313 - exemplos/04_conversoes

8. Expressões booleanas e curto-circuito

As expressões relacionais e booleanas são utilizadas para determinar condições de execução. Um ponto interessante é que linguagens diferentes podem tratar valores e comparações de maneiras distintas. Em C, valores inteiros podem ser utilizados como condições, sendo 0 considerado falso e valores diferentes de zero considerados verdadeiros.

O exemplo `3 > 2 > 1` em C resulta em 0. Primeiro `3 > 2` é verdadeiro e assume o valor 1; depois a expressão passa a ser `1 > 1`, que é falsa. Em Python, entretanto, a mesma construção representa uma comparação encadeada equivalente a `(3 > 2) and (2 > 1)`, resultando em True.

Python e JavaScript também possuem o conceito de truthiness, no qual diversos valores podem ser interpretados como verdadeiros ou falsos em uma condição. Valores como 0, string vazia, listas vazias e objetos vazios podem ser tratados como falsos.

Java, C#, Go e Rust exigem uma expressão booleana explícita em condições.

O curto-circuito permite determinar o resultado de uma expressão lógica sem avaliar todos os seus operandos. Em Java, por exemplo, && evita que a segunda condição seja avaliada quando a primeira já determina o resultado. Isso pode impedir acessos inválidos e exceções. O operador & não possui o mesmo comportamento de curto-circuito.

Em JavaScript, o operador ?. permite acessar propriedades de maneira segura quando um valor é nulo, enquanto ?? permite definir um valor alternativo somente quando o valor original é null ou undefined. Isso é diferente do uso de ||, que considera 0 e outros valores falsy. O exemplo qtd || 10 transforma 0 em 10, enquanto qtd ?? 10 preserva o zero.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0  (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": `and` e `or` devolvem um dos **operandos**.

Sebesta, 11ª ed. · §7.6 · p. 315-317 · exemplos/05_booleans

9. Sentenças de atribuição

A atribuição é responsável por armazenar valores em variáveis e também apresenta diferentes características dependendo da linguagem. Em C, C++ e JavaScript, a atribuição pode ser tratada como expressão, permitindo construções como `sum = count = 0` e `while ((ch = getchar()) != EOF)`. Essa flexibilidade também pode criar erros.

Um exemplo clássico é escrever `if (x = y)` quando a intenção era comparar `x` e `y`. Em C, a construção pode compilar porque a atribuição produz um valor. Java e C# reduzem esse risco ao exigir um booleano no `if`. Python não considera a atribuição comum uma expressão, embora possua o operador morsa `:=` para casos específicos.

Existem ainda atribuições compostas, como `+=`, e operadores de incremento, como `count++` e `++count`. Python permite atribuição múltipla, sendo possível trocar dois valores com `a, b = b, a`. Em linguagens funcionais, a ideia pode ser diferente: estruturas como `val` e `let` dão nome a valores sem modificar o valor original.

A atribuição de modo misto mostra outra diferença importante. Java permite alargamento automático, mas exige cast explícito para estreitamento em atribuições comuns. C aceita coerções em várias direções, podendo transformar valores silenciosamente e provocar perdas.

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;      // 3 (cast explícito)
long l = i;             // ok

i += 3.7;               // COMPILA: cast escondido
System.out.println(i);  // 6
```

C · coerção em qualquer direção

```
int i = 3.7;            // 3, em silêncio
char c = 300;           // 44 (só um aviso)
unsigned int u = -1;     // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para `double`.

Sebesta, 11ª ed. · §7.8 · p. 321-322 · exemplos/06_atribuicao

10. Estruturas de controle

As estruturas de controle determinam a ordem em que as sentenças de um programa são executadas. O material organiza as estruturas fundamentais em sequência, seleção

e iteração. A sequência executa uma sentença após a outra; a seleção escolhe um caminho; e a iteração repete sentenças enquanto uma condição for satisfeita.

Böhm e Jacopini demonstraram em 1966 que sequência, seleção e laço lógico são suficientes para representar algoritmos. O goto, portanto, não é necessário para expressar algoritmos. Mesmo assim, linguagens oferecem estruturas como for, switch e do-while porque elas facilitam a escrita e podem tornar determinados algoritmos mais claros.

O desafio está em equilibrar facilidade de escrita e legibilidade. Quanto maior a quantidade de estruturas diferentes em uma linguagem, maior pode ser a dificuldade de leitura de código escrito por pessoas que conhecem apenas uma parte dessas estruturas.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };
y = x if x > 0 else 2 * x
```

// Rust: if é uma expressão (como em F#, p. 334)
Python: expressão condicional (p. 306)

Sebesta, 11ª ed. · §8.2.1 · p. 330-334 · exemplos/07_selecao

11. Seleção

A seleção de dois caminhos normalmente é representada pelo if e pelo else. Um dos problemas estudados é o chamado "else pendente". Em C, Java, C++ e C#, quando

existem ifs aninhados, o else é associado ao if mais próximo que ainda não possui um else. A indentação visual não altera essa regra.

O problema pode ser reduzido pelo uso de chaves. Go e Rust tornam as chaves obrigatórias nesse tipo de construção, enquanto Python e F# utilizam a indentação como parte relevante da estrutura da linguagem.

A aula também apresenta seleção múltipla por meio de switch. Em C, esquecer o break pode fazer o fluxo continuar para os próximos cases. Java moderno oferece a sintaxe com `->`, que evita esse comportamento e permite utilizar switch como expressão. Go encerra cada case automaticamente, e Rust utiliza match com exigência de cobertura dos casos.

A ideia de exaustividade é importante em Rust: se uma possibilidade não for tratada, o código pode não compilar. Isso transfere parte da verificação para o compilador e pode reduzir determinadas classes de erro.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

12. Iteração

A iteração permite repetir operações. O `for` em C é apresentado como um laço lógico bastante flexível, no qual inicialização, condição e atualização podem ser controladas pelo programador. O exemplo mostra que modificar a variável de controle dentro do corpo altera a quantidade de passos e, conseqüentemente, a saída.

Python possui uma abordagem diferente: seu `for` percorre um objeto iterável. No exemplo com `range`, modificar a variável dentro do corpo não altera os valores que serão fornecidos nas próximas iterações. Isso demonstra que estruturas com aparência semelhante podem ter semânticas diferentes.

Também são estudados `while` e `do-while`. O `while` é um laço de pré-teste, enquanto o `do-while` executa o corpo pelo menos uma vez. O material alerta que essa característica do pós-teste pode ser esquecida pelo programador.

O caso dos tocadores Zune demonstra o risco de um laço sem progresso. Em um determinado dia bissexto, a variável `dias` permanecia com valor 366, mas nenhuma das condições alterava esse valor. Como a condição do `while` continuava verdadeira, o programa não conseguia sair do laço.

`Break` e `continue` permitem controlar a execução de laços. Java também permite `break` rotulado, que pode encerrar laços aninhados. No JavaScript, é necessário distinguir `for...in` de `for...of`: o primeiro percorre as chaves e o segundo percorre os valores. Confundir os dois pode gerar resultados inesperados.

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + ", " + j);
      break externo; // sai dos DOIS laços
    }
```

JavaScript · for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. · §8.3.2-8.3.4 · p. 345-352 · exemplos/08_iteracao

13. Goto e comandos protegidos

O goto é apresentado como a estrutura de desvio incondicional mais poderosa. Justamente por isso, seu uso sem restrições pode prejudicar a legibilidade e a manutenção. O material relaciona essa discussão ao texto de Dijkstra de 1968, “Go To Statement Considered Harmful”.

O exemplo do “goto fail” mostra como uma segunda instrução goto, colocada sem chaves, podia ser executada sempre. A consequência era pular uma etapa de verificação. O caso demonstra novamente a relação entre indentação visual, estruturas de controle e comportamento real do compilador.

Atualmente, C, C++, C# e Go possuem goto, enquanto Java não oferece o comando como construção utilizável. Estruturas como break, continue e return podem ser entendidas como formas restritas de desvio e, em muitos casos, oferecem maior clareza.

Por fim, a aula apresenta os comandos protegidos de Dijkstra. Nessa abordagem, várias guardas são avaliadas e, quando mais de uma é verdadeira, uma delas pode ser

escolhida de maneira não determinística. A ideia está relacionada à verificação de corretude e influenciou mecanismos posteriores, incluindo o select da linguagem Go.

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do **CSP** de Hoare (1978) e do select de Go

Go - select, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Sebesta, 11ª ed. · §8.5 · p. 353-355 · exemplos/09_desvio_protegido

14. Exercício prático e análise dos bugs

A parte final da aula propõe um exercício em duplas com seis pequenos programas. A atividade pede que o estudante determine a saída, identifique a regra responsável pelo comportamento e proponha uma correção. Dessa forma, o exercício reúne os principais assuntos estudados.

No primeiro caso, em JavaScript, o problema está no uso de for...in para percorrer um array. As chaves são strings, e a soma acaba realizando concatenação, produzindo "0012". A correção é utilizar for...of ou outra forma de percorrer os valores.

No segundo caso, em Python, a expressão $d = d \text{ or } 10$ considera o valor 0 como falso. Como 0 era um desconto válido, o programa substitui o valor recebido por 10 e calcula um resultado diferente do esperado. A correção indicada é testar especificamente se d é None.

O terceiro caso utiliza switch em Java. Como os cases não possuem break, o fluxo continua até o default e o resultado final passa a ser “inválido”. A solução pode ser adicionar break ou utilizar a sintaxe moderna do switch com seta.

O quarto caso apresenta um ponto e vírgula logo após um if em C. O ponto e vírgula forma uma sentença vazia, fazendo com que o printf seguinte seja executado independentemente da condição. O uso de chaves e avisos do compilador ajuda a evitar esse tipo de problema.

No quinto caso, em Go, o operador := dentro do if cria uma nova variável x, causando sombreamento. Assim, o valor dentro do bloco é 20, mas o valor fora continua 10. A correção é utilizar x = x * 2.

No sexto caso, em Java, acertos / total realiza uma divisão inteira antes da atribuição ao double. Como 7 / 10 resulta em 0, a taxa final é 0.0%. A correção é realizar o cálculo com um valor real, como acertos * 100.0 / total.

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 - Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 - C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

15. Síntese dos principais aprendizados

A aula mostra que muitos bugs surgem de pequenas diferenças entre a intenção do programador e as regras da linguagem. Precedência inadequada ou desconhecida pode mudar o resultado de uma expressão; coerções automáticas podem esconder erros de tipo; efeitos colaterais podem tornar a ordem de avaliação relevante; e estruturas de controle mal utilizadas podem criar caminhos de execução inesperados.

Também fica evidente que as linguagens modernas incorporaram mecanismos para evitar determinadas classes de problemas. Go e Rust, por exemplo, exigem conversões explícitas em situações que outras linguagens tratam automaticamente. Rust utiliza match exaustivo, Go e Rust exigem chaves em estruturas de seleção e Java moderno possui uma forma de switch sem o comportamento tradicional de queda entre cases.

A principal ideia é que uma linguagem de programação não é apenas uma coleção de comandos. Suas regras de avaliação, conversão, seleção e iteração influenciam diretamente a confiabilidade e a legibilidade do software. Conhecer essas regras permite escrever código mais previsível e também compreender melhor código produzido por outras pessoas.

Atividades complementares (autopesquisa)

- **Expressões em Ruby e em Lisp** (§7.2.1.4-7.2.1.5, p. 305-306):
`expressoes.rb`, `expressoes.lisp`
- **Expressões condicionais** (§7.2.1.6, p. 306) e **erros em expressões** (§7.4.3, p. 313)
- **Implementação da seleção múltipla**: tabelas de desvio (§8.2.2.3, p. 337-339)
- **Laços em linguagens funcionais**: contagem por recursão (§8.3.1.4, p. 345)
- **Iteradores e blocos de Ruby** (§8.3.4, p. 349-352)
- Leitura: Dijkstra, "Go To Statement Considered Harmful" (1968)

Entrega sugerida

Um mesmo exemplo com switch em C, switch -> em Java, switch em Go e match em Rust: compare "cair" entre casos, exaustividade e legibilidade.

Próxima aula

Subprogramas (Caps. 9 e 10)

16. Conclusão

A Aula 07 apresentou uma visão ampla sobre expressões, atribuição e controle de fluxo, relacionando conceitos teóricos com exemplos em diferentes linguagens. A utilização de casos reais e exercícios práticos tornou possível perceber que detalhes de sintaxe e semântica podem provocar resultados inesperados e até problemas relevantes em sistemas.

Entre os conteúdos mais importantes estão precedência e associatividade, ordem de avaliação, efeitos colaterais, sobrecarga de operadores, conversões e coerções, expressões booleanas, curto-circuito, atribuições, seleção e iteração. A discussão sobre goto e comandos protegidos complementou o conteúdo ao tratar da organização do fluxo e da busca por estruturas mais legíveis.

Como resultado, a aula reforça uma competência essencial para a Engenharia de Software: não basta saber o que um código parece fazer; é necessário conhecer as regras da linguagem que determinam o que ele realmente faz. A comparação entre linguagens evidencia que decisões de projeto podem privilegiar flexibilidade, simplicidade, desempenho, legibilidade ou confiabilidade, e que o programador deve considerar essas características ao desenvolver e revisar software.